Selecting Reliable LLM Reasoning with Numeric-Answer Groups and Local Arithmetic Checks

Wu Houlin | Student ID: 3036863317

DASE7506 Mini-project 2 | Revised 27 October 2026

Abstract

This project tests whether checking calculations helps select among four supplied solutions to a maths problem. The selector groups numerically equivalent answers, scores them using the supplied embeddings and six question/answer features, then adds a learned correction from local arithmetic checks. It selects **396/467 correct answers (84.7966%)** on the complete official test bank. The checkpoint occupies 9,157 bytes, and fresh CPU prediction plus scoring took 1.383 seconds locally. Compared with the model without arithmetic features, it answers 18 more hard development questions correctly in ordinary five-fold evaluation and 22 more in a lexical-cluster split. Several extensions scored slightly higher on development data but failed the improvement thresholds set for their experiments. Larger score gaps identify more accurate subsets, although consistent calculations can still produce the wrong requested quantity.

1. Task and evaluation populations

Each question provides four fixed candidate solutions and a 1,024-dimensional representation for each candidate. The output is one candidate index. The official scorer checks that candidate's parsed final number; it does not certify every reasoning step [1]. No external datasets, additional pretrained weights, or new LLM generations are used.

Population	Questions	Role
Supplied training split	4,755	All final fitting
Training development pool	3,804	Fixed five-fold comparisons
Development questions with conflicting answers	1,691	Main development accuracy denominator
Training holdout excluded from development folds	951; 423 hard	Additional checks; previously used for selection
Official validation	798	Development selection and reproduction
Official test	467	Frozen final evaluation; all are hard

A hard question has at least two different valid parsed numeric answers. In the development pool, the other 2,113 questions have one valid answer group and are automatically correct when that group is selected. Reporting hard accuracy prevents this agreement from hiding ranking errors. Validation and test percentages therefore describe different populations.

2. Research questions

The main hypothesis is that local calculation checks add useful information beyond embeddings and answer formatting. A format-only control tests that explanation, and a second split checks how much the result depends on the original folds. The reliability analysis asks whether larger score gaps identify more accurate choices. Learned verifiers [2] motivate ranking existing solutions. PAL [3] motivates executing calculations, while step-level supervision [4] motivates examining intermediate work. The present method uses hand-written checks and final-answer labels; it does not use the models or additional datasets from these papers.

3. Final selection method

3.1 Numeric-answer groups and the surface anchor S

The unchanged course parser represents equivalent fractions and decimals with a canonical numeric string. Valid candidates with the same answer form one group. Members are ordered by text and float64 embedding bytes before pooling; ties depend on content rather than IDs or candidate position. Invalid parsed answers are excluded when a valid answer exists.

For group g, S uses the mean 1,024-dimensional embedding, support count, support divided by four, number of answer groups, and scalar embedding variance. Six additional features describe answer sign, clipped log magnitude, integer status, exact occurrence among retained question numbers, retained-number count, and answer magnitude relative to the largest retained absolute question number. This yields 1,034 coordinates. Question-number extraction retains at most 16 bounded numbers from the first 20,000 characters. The ratio uses a denominator of one if no nonzero question number is retained.

S is a no-intercept logistic preference model with mean pair loss plus 0.005 times the squared standardized coefficient norm. It uses lbfgs, maxiter=2000 and tolerance 10^-6. During training, every nonempty subset within each answer group is available for correct-versus-incorrect comparisons. Both pair directions are included, with equal total loss weight for each informative question. Scalers are fitted on original complete groups, including one-answer questions; augmented subsets do not fit preprocessing. At inference, only the original complete groups are used.

3.2 Local arithmetic residual L

Each candidate contributes the following 11 features. Group means and maxima produce 22 residual coordinates. Log transforms below use log(1+x).

Block	Candidate features
Format, 3	Any equals sign; log equals-sign count; log count of lines containing equals signs
Arithmetic, 8	Any checked equality; log check count; log mismatch count; mismatch fraction; log maximum relative residual; final/right-hand-result comparison available; agrees; disagrees

Format counts inspect the full text. Arithmetic checks normalize at most the first 20,000 characters and visit at most 64 equals-sign opportunities. A whitelisted interpreter evaluates closed numeric expressions with exact fractions. Expressions are limited to 160 characters and 64 AST nodes; magnitudes are bounded by 10^12, numerators and denominators by 10^18, and integer exponents by absolute value six. Unsupported syntax contributes missing information. No Python eval or exec is used.

An equality is accepted within absolute tolerance 0.005 plus relative tolerance 10^-6 times the larger operand magnitude. The last checked right-hand result is compared with the final answer parsed from the full original text. These are soft features: an internal mismatch does not force rejection.

For each outer fold, S is frozen and a zero-initialized residual is fitted on original full-group pairs only. For an ordered pair (a,b), let delta_z=z(a)-z(b) for standardized group features, d=S(a)-S(b), and t=+1 when a is correct, otherwise -1. The objective is:

```
weighted_mean softplus(-t * (d + delta_z @ v))
    + 0.005 * ||v||^2,       t in {+1, -1}
final group score = S(group) + raw_features(group) @ raw_v
```

The residual uses L-BFGS-B with maxiter=2000, ftol=10^-12, gtol=10^-6 and maxls=40. Scaling is absorbed into exported weights. The final checkpoint contains 1,034 anchor coefficients and 22 residual coefficients, all float64.

4. Development design and core ablations

The training partition and ordinary five-fold splits were fixed with seeds 20261021 and 20261022. In each fold, weights and scalers are fitted using that fold's training rows. Out-of-fold (OOF) scores therefore come from models that excluded the scored rows during fitting. All comparisons below use the same folds and parsed-answer groups.

Core method	Hard correct / 1,691	Accuracy	Fixed / broken vs S
Group-only C1028, with subset training	1,370	81.02%	Not compared here
S: group features plus numeric surface	1,392	82.32%	0 / 0
S plus format-only residual	1,396	82.55%	14 / 10
L: S plus format and arithmetic residual	1,410	83.38%	33 / 15

L improves on format-only by 14 correct questions, fixing 26 and breaking 12. Its fold gains over S are [2, 3, 4, 6, 3]. The format-only residual improves S by four questions, compared with 18 for the full residual. Counting equation formatting therefore does not explain the full improvement. The arithmetic block also checks final-answer compatibility and adds more coefficients, so this comparison cannot attribute the gain solely to detecting incorrect intermediate calculations.

4.1 Checking the result with a different split

A second five-fold design groups development questions by input-only lexical clusters: TF-IDF with at most 2,048 features, 32-component SVD and 64-cluster K-means, using seed 20261023; StratifiedGroupKFold uses seed 20261024. No cluster crosses a fitting/validation boundary. The split construction sees all development input questions, but model preprocessing still fits only each fold's training rows. These lexical clusters approximate related wording, not known mathematical templates.

Evaluation	S	L	Net correct gain
Ordinary five-fold hard / 1,691	1,392	1,410	+18
Lexical-grouped five-fold hard / 1,691	1,383	1,405	+22
Previously queried training holdout / 423 hard	346	347	+1
Official validation / 798	746	751	+5

On the grouped split, L fixed 34 and broke 12 S decisions. Before this comparison, the confirmation plan required at least five net additional correct answers and at least three folds where accuracy did not decrease; L passed. The training-holdout gain was only one question, with five fixes and four breaks. This small difference gives little evidence of improvement beyond the development pool. Both the holdout and official validation had already been used for model selection.

4.2 Answer diversity

For questions with two, three and four distinct answers, S versus L correct counts were respectively 954/1,081 versus 957/1,081; 350/479 versus 359/479; and 88/131 versus 94/131. More distinct answers were harder to rank. The largest percentage gain occurred in the smallest, four-answer subgroup; its sample size limits interpretation.

5. Why further extensions were not selected

These extensions used the same 1,691 hard development questions. For most later experiments, the proposed replacement needed at least ten additional correct answers over L, at least five over a matched control, and at least three folds without a decrease against each comparison. Each round's criteria were set before its results were inspected. These practical thresholds limited model replacement, but they are not statistical significance tests and do not remove the risk of overfitting through repeated experiments.

Extension	Hard correct	Interpretation of the recorded comparison
Quantity bag / canonical order / temporal order	1,409 / 1,414 / 1,409	The planned temporal-order method did not meet the criteria
Cross-fitted S offsets	1,405	Five below L and its matched control
Joint surface/arithmetic subset training	1,410	Tied L; 17 fixes and 17 breaks
Refitted affine consistency plus final matching	1,416	+6 over L, +4 over coverage control; below both thresholds
Consistency-only / final-match-only ablation	1,416 / 1,415	Follow-up ablation, not a new selection experiment
Frozen-L affine correction	1,415	+5 over L, +3 over coverage control; below both thresholds

The full affine model combined contradiction checks and final-answer matching. Its coverage control recorded whether checks were available, so that comparison did not isolate the two components. In the follow-up ablation, consistency-only matched the full model's total accuracy, but four selected numeric answers changed: two became correct and two became wrong. Its one-question advantage over final-match-only is too small to support a strong preference between them.

Freezing L's existing weights preserved its exact scores on 2,994 questions with no new affine features, including 1,324 hard questions. The added correction therefore left those questions unaffected. It still missed the required gains, so L remained the final choice despite some extensions having slightly higher development accuracy.

5.1 A substitution rule with too few usable cases

A final input-only probe checked multivariate affine expressions using nearby singleton assignments within a paragraph and an eight-line joint span. It found 934 multivariate records, but 914 lacked the required assignments. Only 20 substitution checks remained, including 14 inconsistencies. Just five hard questions had a usable check, and five had varying consistency evidence across answer groups. This failed the fixed 100/30 opportunity thresholds, so no model was fitted. The counts measure availability under this bounded rule, not answer correctness, statistical power or the usefulness of general system solvers.

A separate checking script reconstructed all 15,216 development candidate traces and their features for this probe. Unsupported expressions and uncertain scopes counted as missing evidence. They were not labelled incorrect.

6. Reliability and failure analysis

For each hard OOF question, the score gap is the highest group score minus the second-highest score for a different numeric answer. Duplicate candidates for one answer do not create a runner-up. The gap is a ranking margin, not a calibrated probability. The following post-selection analysis uses existing predictions without fitting a confidence model.

Largest-gap fraction retained	L correct / retained	Conditional accuracy
All hard questions	1,410 / 1,691	83.38%
Approximately 90%	1,324 / 1,522	86.99%
Approximately 75%	1,159 / 1,269	91.33%
Approximately 50%	811 / 846	95.86%
Approximately 25%	420 / 423	99.29%
Approximately 10%	170 / 170	100.00%

The code rounds the requested subset size up and retains all ties at the cutoff, so coverage can slightly exceed the requested fraction. These accuracies describe only the retained questions and cannot replace the complete-bank score. The 170/170 result is an observation on this development subset, not a guarantee for future questions. L's pooled gap AUROC for distinguishing correct from wrong choices is 0.81328, almost unchanged from S's 0.81322; its fold AUROCs range from 0.80001 to 0.83655. Pooled gaps also come from separately fitted fold models whose score scales need not be equally calibrated.

S and L selected different numeric answers on 53 hard questions: L alone was correct on 33, S alone on 15, and both were wrong on five. Of the 1,638 agreement cases, 261 were identical wrong answers. Thus both models were wrong on 266 questions in total. A hypothetical oracle choosing between their two existing answers could add only 15 correct choices to L; it requires correctness labels and is not an implementable result.

6.1 Internally valid arithmetic can answer the wrong quantity

One high-gap development error asks how old a five-year-old child will be when his 32-year-old father is twice his age. The selected candidate correctly solves:

```
32 + t = 2 * (5 + t)  =>  t = 22 years elapsed
requested age = 5 + 22 = 27, rather than 22
```

It then reports 22 as the child's age. The selection gap was 4.2566. Its algebra is compatible with elapsed time, while the final quantity is wrong. This illustrates a limitation of local calculation checks: they cannot reliably connect a computed quantity to the question's semantic target. Conversely, a correct final number does not certify every sentence of a solution. This example was selected from the development diagnostics, not from test errors.

7. Final test, resources and reproducibility

The final L method was selected and its source/checkpoint hashes committed before its test score was read. It achieved **396/467 = 84.79657387580299%** with the unchanged official predictor and independent scorer. The historical published S baseline scored 386/467 (82.6552%). Their aggregate difference is ten correct questions, or 2.1413 percentage points. No per-question test-error analysis, parameter adjustment or model switching followed the L test result.

Aggregate test scores from earlier baselines were already known during the later research. Although L was chosen before its own test evaluation, that history and the repeated development comparisons limit claims about generalization. The reported accuracy applies to this course bank.

Resource or reproduction check	Recorded result
Local CPU	Intel Core i5-14600KF; numerical libraries limited to four threads
Selected training device	CPU; no GPU required
Active inference assets	9,157 bytes; 18,246 including unused historical S checkpoint
Fresh complete test prediction + scoring	1.383 s locally, including startup/imports/loading/preprocessing
Fresh complete validation reproduction	751/798; 1.405 s in a NumPy-only environment
Clean full training reproduction	4,755 rows; two fits; 10.359 s; exact checkpoint bytes
Clean five-fold reproduction	Ten fits; 33.118 s summed fitting time; every S/L OOF score identical

The local timing is not a measurement on the course's reference Xeon Platinum 8457C. Reference-machine eligibility remains subject to the official 30-second rule. Training durations exclude initial input loading; test timing covers the full cold pipeline. Inference needs NumPy 2.3.5; training also pins SciPy 1.16.3, scikit-learn 1.7.2 and threadpoolctl 3.6.0, with Python 3.12.14 used locally. Seeds are stored with the split protocol.

7.1 Search cost and verification scope

Recorded late-stage round costs include quantity-flow controls (30 fits, 134.35 s), cross-fitted offsets (35 fits, 124.65 s), affine checks (20 fits, 121.22 s), and frozen-L corrections (ten new fits, 139.75 s). These round durations include different guard/scoring workloads and are not comparable per-fit benchmarks. An earlier numeric-surface search recorded 95 CV optimizer fits. The selected-pipeline clean reproduction used 12 fits; reproducing C/S/format controls added 15 more. These are verification work, not the total search budget. A unified wall-clock total across the early exploratory history was not retained.

Separate verification scripts reconstructed training pairs, scalers, objectives and gradients. Clean full training reproduced both checkpoint files byte for byte; 50 fold-array artifacts and pooled OOF scores also matched the archived versions. Inference checks covered checkpoint reload, changed IDs, candidate permutation, empty input, Python-integer outputs, blocked reference reads and blocked non-checkpoint array reads. These were automated checks developed with Codex assistance, not an independent human review. Raw runs are excluded from the public repository.

8. Reproduction, limitations and disclosure

The README gives dependency versions, the pinned data source and commands for each experiment. With that environment active, these PowerShell commands train the selected model, reproduce the core comparisons, then evaluate the newly trained checkpoint. To evaluate the distributed checkpoint instead, omit the --checkpoint argument. The reliability-analysis command is in the README.

```
$data = 'upstream/code/data'
python src/train.py --data-dir $data --output runs/retrained
python src/cross_validate.py --data-dir $data --output runs/cv
python src/reproduce_controls.py --data-dir $data --output runs/controls
python src/evaluate.py --data-dir $data --split test `
    --checkpoint runs/retrained/checkpoint --output runs/retrained-test
```

Only supplied training rows fit weights or preprocessing. Evaluation is offline and uses the caller's bank, embeddings and frozen assets. IDs align records but never enter scoring. Code, dependency versions, aggregate official JSON, checkpoint hashes, split definitions, the compact research registry and AI disclosure are provided. The final publication commit and matching checkpoint links are identified in the release and submission record.

The scripts reproduce the selected method, core C/S/format ablations and OOF reliability analysis. The other extensions are documented in experiment summaries with configuration details and source hashes. Their full implementations and raw runs remain in the local research workspace, so those extensions cannot all be rerun from this submission package alone.

Local arithmetic checks improved the selected answer count, but their coverage is narrow. They rely on equation syntax and cannot reliably identify the quantity requested by a question. The model can also rely on formatting or correlations in the supplied embeddings. The one-question holdout gain and repeated reuse of development data are reasons to treat the improvement as modest. An unseen audit bank would provide a stronger test than further small gains on the same development pool.

AI assistance and reused work

OpenAI Codex provided substantive assistance with experimental design, code, execution, verification scripts and report drafting and revision. Local instructions from Scientific Agent Skills [5] supported experiment management and writing checks; Humanizer instructions supported language editing. These automated checks do not replace the student's responsibility to understand and review the code and claims. The course data, numeric parser, predictor and scorer are reused with their attribution and license notices. References [2-4] motivate the design; their additional datasets and pretrained weights are not used.

References

[1] DASE7506 MP2. Selecting Reliable LLM Reasoning: Project Guide. Accessed 27 October 2026. https://github.com/xudongwu-0/DASE7506-MP2/blob/main/guide/GUIDE.md

[2] Cobbe, K. et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021. https://arxiv.org/abs/2110.14168

[3] Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., and Neubig, G. PAL: Program-aided Language Models. ICML, PMLR 202:10764-10799, 2023. https://proceedings.mlr.press/v202/gao23f.html

[4] Lightman, H. et al. Let's Verify Step by Step. arXiv:2305.20050, 2023. https://arxiv.org/abs/2305.20050

[5] Kassis, T., Agarwal, V., He, Y., Patel, D., and Brueckner, A. M. Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents. arXiv:2609.00065, 2026. https://doi.org/10.48550/arXiv.2609.00065